204252 Data Structures and Analysis

Chapter 7: Priority Queues (Binary Heaps)

Thapanapong Rukkanchanunt and Jakarin Chawachat

August 6, 2026

Computer Science Department

Outline

Priority Queue ADT

Binary Heap Properties

Heap Operations

Applications

Priority Queue ADT

The Priority Queue Model

A **Priority Queue** is an Abstract Data Type (ADT) that maintains a collection of elements, each with a priority (or key), and supports:

- `insert(x)`: Adds an item x to the queue.
- `deleteMin()`: Removes and returns the element with the *smallest* key (highest priority) in a Min-Priority Queue.



Comparison and Applications

Container Comparison

- **Stack**: First-In, Last-Out (FILO) $\rightarrow$ Accesses newest element.
- **Queue**: First-In, First-Out (FIFO) $\rightarrow$ Accesses oldest element.
- **Priority Queue**: Priority-First $\rightarrow$ Accesses minimum/maximum element.

Real-World Applications

- **OS Scheduler**: Selects process with highest priority/shortest remaining job time.
- **Network Router**: Prioritizes voice/video packets over normal bulk traffic.
- **Graph Algorithms**: Prim's MST, Dijkstra's Shortest Path, Huffman Coding.

Simple Implementations: Cost Analysis

What if we implement a Priority Queue using standard sequential lists?

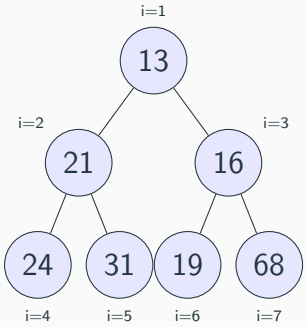
Data Structure	`insert(x)`	`deleteMin()`
Unsorted Array / List	$\mathcal{O}(1)$	$\mathcal{O}(N)$ (scan for min)
Sorted Array / List	$\mathcal{O}(N)$ (shift elements)	$\mathcal{O}(1)$ (remove first)
Binary Search Tree	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$
Binary Heap (Textbook)	$\mathcal{O}(\log \mathbf{N})$ (worst)	$\mathcal{O}(\log \mathbf{N})$

- BST is overkill because it keeps all elements fully sorted and requires expensive pointers.
- A **Binary Heap** is a specialized, pointer-free complete binary tree that performs both operations in worst-case $\mathcal{O}(\log N)$ time.

Binary Heap Properties

Structure Property: Complete Binary Tree

A heap is a **Complete Binary Tree**: every level is filled except possibly the bottom level, which is filled from left to right.



Array Representation: We store the elements in a simple array starting at **index 1**:

- **Left Child** of i: $2i$
- **Right Child** of i: $2i + 1$
- **Parent** of i: $\lfloor i/2 \rfloor$
- **Index 0**: Sentinel/Dummy

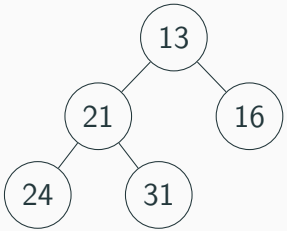
i	0	1	2	3	4	5	6	7
val	–	13	21	16	24	31	19	68

Heap-Order Property

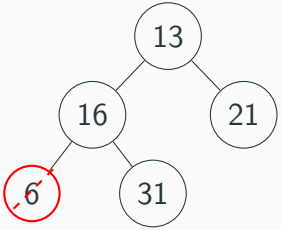
Min-Heap Ordering Rule

For every node X (except the root), the key in the parent of X is less than or equal to ($\leq$) the key in X.

Valid Min-Heap



Invalid Heap (Order violation)



Heap Operations

Insertion: Percolate Up

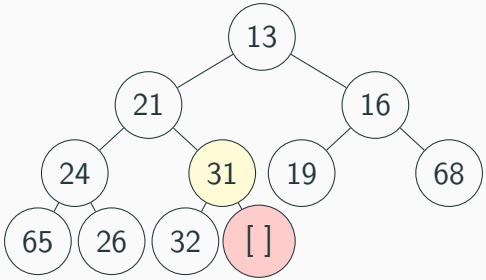
To insert an element x:

1. Create a "hole" at the next available location in the complete tree.
2. If x can be placed in the hole without violating heap-order, place it there.
3. Otherwise, slide parent down into the hole (bubbling the hole up) and repeat.

Insertion Step-by-Step (Inserting 14)

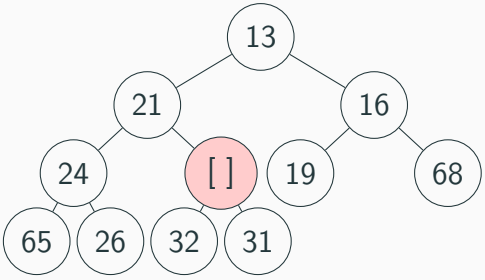
Step 1: Create Hole

- To insert **14**, we create a hole at index 11 (next available position).
- Compare 14 with parent **31** (index 5). Since $14 < 31$, we bubble up.



Step 2: Percolate Up

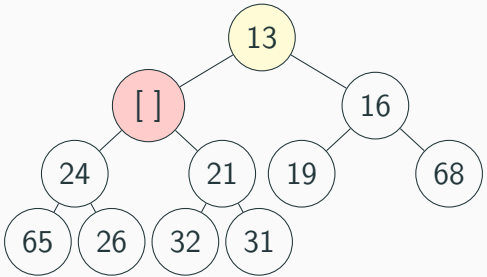
- Slide **31** down into the hole. The hole moves to index 5.
- Compare 14 with parent **21** (index 2). Since $14 < 21$, bubble up again.



Insertion Step-by-Step (Inserting 14) Cont.

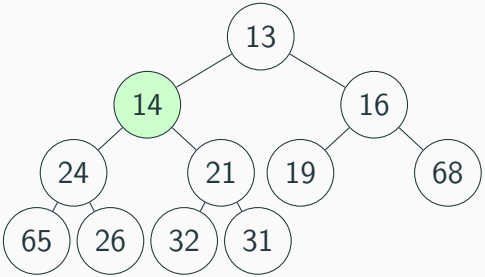
Step 3: Percolate Up

- Slide **21** down into the hole. The hole moves to index 2.
- Compare 14 with parent **13** (index 1). Since $14 > 13$, the hole stops here.



Step 4: Final Placement

- Place **14** into the hole at index 2.
- The heap-order property is now fully restored.



Insertion Implementation

Weiss's implementation copies the inserted item to index 0. This functions as a **sentinel value** to stop the loop without an explicit 'hole ¿ 1' test.

```
1  // Insert item x, allowing duplicates.
2  void insert( const Comparable & x ) {
3      if( currentSize == array.size( ) - 1 )
4          array.resize( array.size( ) * 2 );
5
6      // Percolate up
7      int hole = ++currentSize;
8      Comparable copy = x;
9      array[ 0 ] = std::move( copy ); // Sentinel
10
11     for( ; x < array[ hole / 2 ]; hole /= 2 )
12         array[ hole ] = std::move( array[ hole / 2 ] );
13     array[ hole ] = std::move( array[ 0 ] );
14 }
```

Insertion Implementation

- **Complexity**: $\mathcal{O}(\log N)$ worst case. On average, only 2.6 comparisons are needed, so the average insertion takes $\mathcal{O}(1)$ time.

Deletion: Percolate Down

To perform `deleteMin()`:

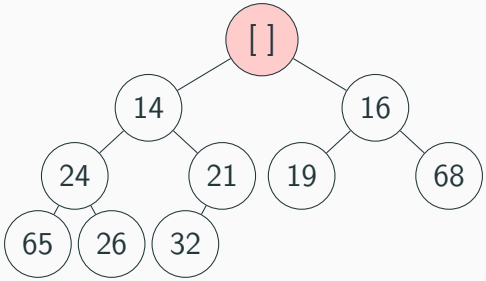
1. Remove the root (which holds the minimum value). This leaves a hole at root index 1.
2. Grab the last element in the heap, *tmp*.
3. Slide the smaller child of the hole up, pushing the hole down.
4. Repeat until *tmp* can fit into the hole without violating heap-order.

Edge Case: Single Child
If the heap has an even number of elements, one node might have only a left child. The algorithm must handle this carefully.

Deletion Step-by-Step (deleteMin)

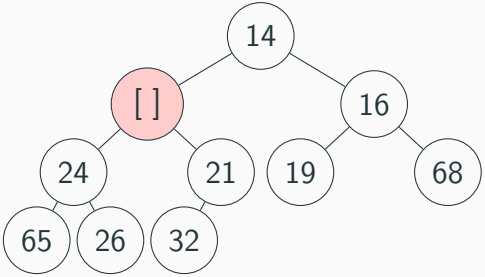
Step 1: Remove Root & Hold 31

- Remove root **13**. The hole is at index 1.
- Temporarily hold the last element **31** (index 11). Size becomes 10.



Step 2: Percolate Down

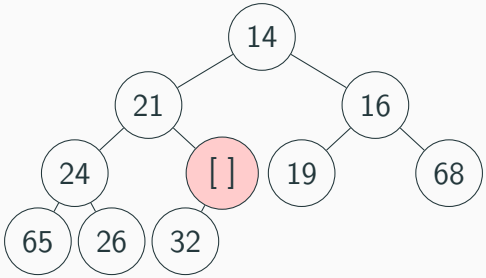
- Compare children **14** and **16**. Smaller is 14.
- Since $31 > 14$, slide 14 up. Hole moves to index 2.



Deletion Step-by-Step (deleteMin) Cont.

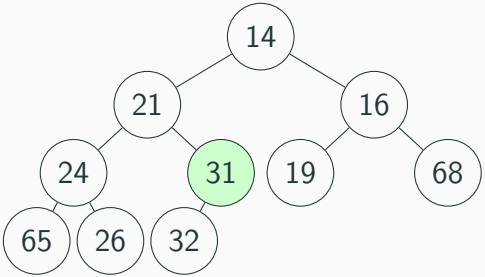
Step 3: Percolate Down

- Compare children **24** and **21**. Smaller is 21.
- Since $31 > 21$, slide 21 up. Hole moves to index 5.



Step 4: Final Placement

- Compare child **32**. Since $31 < 32$, the hole stops here.
- Place **31** into the hole at index 5.



Deletion Implementation

```
1  void deleteMin( ) {
2      if( isEmpty( ) ) throw UnderflowException{ };
3      array[ 1 ] = std::move( array[ currentSize-- ] );
4      percolateDown( 1 );
5  }
6  void percolateDown( int hole ) {
7      int child;
8      Comparable tmp = std::move( array[ hole ] );
9      for( ; hole * 2 <= currentSize; hole = child ) {
10         child = hole * 2;
11         // Check if right child exists and is smaller than left child
12         if( child != currentSize && array[ child + 1 ] < array[ child ] )
13             ++child;
14         if( array[ child ] < tmp )
15             array[ hole ] = std::move( array[ child ] );
16         else
17             break;
18     }
19     array[ hole ] = std::move( tmp );
20 }
```

Applications

The Selection Problem

Given N elements, find the k-th smallest (or largest) element.

Algorithm 7A (k-th Smallest)

- Read N elements into array.
- Call `buildHeap()` $\rightarrow \mathcal{O}(N)$ time.
- Call `deleteMin()` k times $\rightarrow \mathcal{O}(k \log N)$ time.
- **Total Time**: $\mathcal{O}(N + k \log N)$.

Algorithm 7B (k-th Largest)

- Maintain a Min-Heap of size k containing the k largest elements seen so far.
- For each new element: if larger than root, replace root and percolate down.
- **Total Time**: $\mathcal{O}(N \log k)$.

Algorithm 7A Walkthrough ($k = 3$ Smallest)

Find 3rd smallest of: `[150, 80, 40, 30, 10, 70, 110]`

1. **buildHeap**: Rearrange the input array into a min-heap in-place.
   - Min-Heap: `[10, 30, 40, 150, 80, 70, 110]`
2. **1st** `deleteMin()`:
   - Extracts **10** (1st smallest).
   - Heap becomes: `[30, 80, 40, 150, 110, 70]`
3. **2nd** `deleteMin()`:
   - Extracts **30** (2nd smallest).
   - Heap becomes: `[40, 80, 70, 150, 110]`
4. **3rd** `deleteMin()`:
   - Extracts **40** (3rd smallest). **Done!**

Result: 3rd smallest is **40**.

Algorithm 7B Walkthrough ($k = 3$ Largest)

Find 3rd largest of: `[15, 20, 5, 30, 10, 25, 8]`

Maintain a Min-Heap of size $k = 3$ representing the 3 largest elements.

1. **First 3 items** `[15, 20, 5]`: Build heap $\rightarrow$ `[5, 20, 15]` (root is 5).
2. **Process 30**: Since $30 >$ root 5, replace root and percolate down.
   - Heap: `[15, 20, 30]` (root is 15)
3. **Process 10**: Since $10 \leq$ root 15, ignore it.
4. **Process 25**: Since $25 >$ root 15, replace root and percolate down.
   - Heap: `[20, 25, 30]` (root is 20)
5. **Process 8**: Since $8 \leq$ root 20, ignore it.

Result: The root of the final heap is **20**, which is the 3rd largest.

Discrete Event Simulation

In simulations (e.g. customers arriving at a bank), we must process events chronologically.

Tick-by-Tick (Inefficient)

- Advance clock 1 tick at a time.
- Check if any events occur.
- **Drawback**: Running time depends on the time scale (ticks), not the number of events.

Event-Driven (Priority Queue)

- Store upcoming events in a Priority Queue sorted by event time.
- Repeatedly extract minimum event time $\rightarrow$ immediately advance clock to next event.
- **Benefit**: Runs proportional only to the number of events.